Modern Architecture for Quantum Computers
By Nischal Keshav

Abstract:

The following passages will explain a new model of Quantum Computing Architecture. The main two developments will be an Oracle based memory architecture and a clock system. The oracle based memory systems allow us to develop some programs where certain operations in quantum Computation can be performed without the need of internal quantum gates, but purely by external adjustments to the oracle. The study found that an Oracle based system of memory management allowed for higher control of technologies and avoidance of quantum gates. Using this system also gives us much higher performance and allows for a much greater level of parallelism which allows for linear optimization without any new technology. The hypothetical benchmark test we ran also concluded that our architecture had the potential to assist in the use of many algorithms which required a black box function.

Introduction

Quantum computing is at the height of its development currently at the height of its development and its applications currently seem limitless. From material sciences to breaking encryption, quantum computers have been studied and analyzed for all of their uses, but while there exists much work and analysis on applications of quantum computing and a recent rise in studies on actual technology and how it could be used there has been a lack of analysis on the actual architecture that has been often looked over.

While a hand wave at memory and logic management may work when theorizing how one could make programs that would do certain tasks, we must realize that a definitive and productive model of a quantum computing architecture will be necessary if we want to develop productive machines. The following passage is in hopes to produce some model which can be used as a framework for developing the machine. While this is not a physical project the passage will include some refrences to the physical processes which have a large effect on the study.

Design Objectives

1. **Scalable Architecture:** The design we wish to create must be able to maintain efficiency as the amount of qubits and the number of operations increases. The tool mathematicians used for this is O notation, representing the amount of operations or memory slots needed to accommodate a function as the input size increases.
2. **Compatibility:** It must accommodate all functions which the previous theoretical models have been able to do with the same efficiency. Essentially if there is some function which a theoretical quantum

Introduction to the Memory Units

We begin with the memory center of the computing system. It will act like an Oracle, such that when the Memory Unit receives some string of bits it will output some other qubits. We will use the terminology of line to mean qubit as the language will feel a lot more natural. Other than the lines in the memory unit there is also a set of active qubits which are called the active lines. These are where the majority of computations will take place. There are two advantages to this system.

1. The differentiation will allow us to make physical computers with much fewer resources. It is obvious that qubits need to be prepared differently if we want to operate on them. Say we were to commit an operation. If they were not differentiated the qubits which were not to be operated on would have a much higher chance of a phase flip error(De Wolf 2023).
2. Another main advantage of this is there are many functions that we can conduct which would require a quantum operation, with only classical adjustments to the memory Oracle. These include, but are not limited to the Pauli-X gates and SWAP gates[1]. For example with the SWAP gate instead of the regular quantum operation we could switch the address values of the qubits we want to change.

We will discuss these in more detail when we discuss the modeling of the system, but we must move on to how exactly we will use the operations. Since this is not a discussion on the physical aspects of quantum computing we can ignore the physical constraints for now as long as they are physically viable with our current model of quantum computing.

A challenge arises as we discuss a system of operations. Since we are attempting to maintain the quantum state, actually going through the operations can be challenging, because simply flipping a switch would force the qubit to decide a state. Operations in Quantum Computing are unitary operations which are reversible and represented by matrices. These operations can be committed by various techniques and technologies, but the specifics aren't necessary to this paper. In order to maintain the superposition all the operations must occur instantaneously or they may be in a state where the qubit doesn't need to "choose" a state or another. We should take advantage of both of these strategies when we use tools to apply these operations.

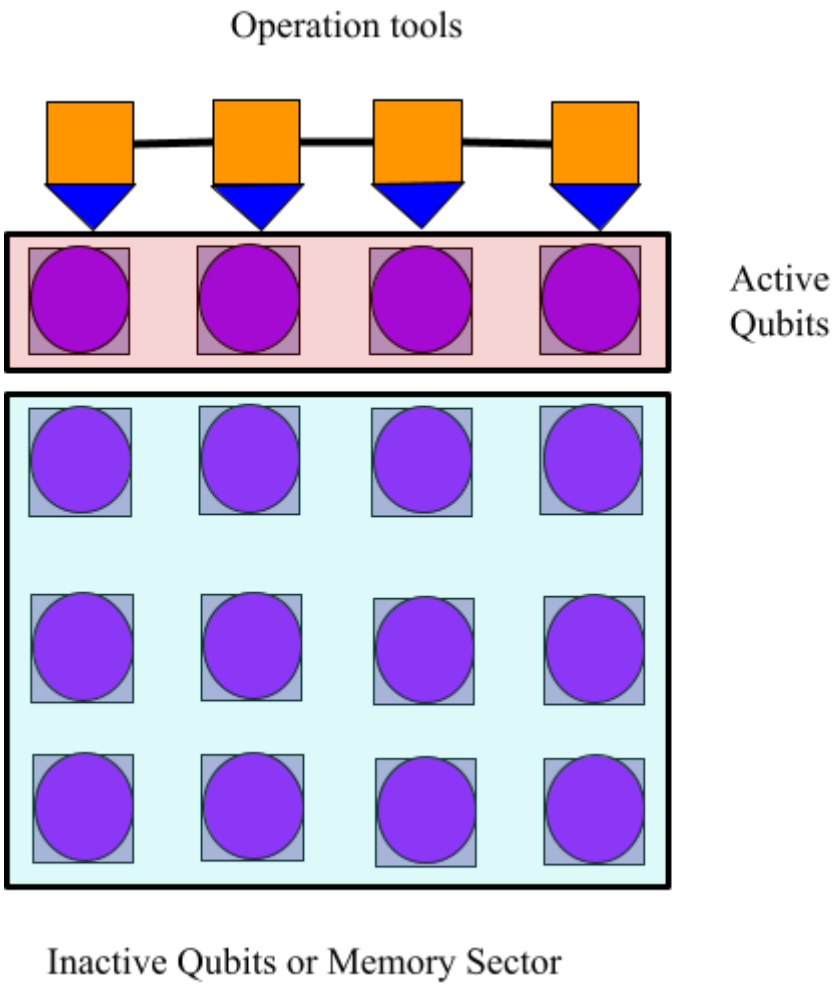


Diagram 1: This diagram shows us the different parts of the quantum component of the quantum computer. The Operation tools are linked together to represent how they often involve multiple qubits. Each of the circles represent a qubit. A SWAP function could represent essentially moving around the qubits. It is also to be noted the Inactive Qubits in the Memory Sector also have operation devices attached, but these will not commit any changes to the inactive qubits.

Error Correction and the Quantum Clock Cycle

Now that we have discussed how memory will be managed, we must go onto how we will analyze how we will use **Error Correction** in these models. Let's create a clock and every clock tick some operations will be applied to the active qubits. We want to apply as many as possible to the qubits at one time as that would allow us to do it in less clock cycles and therefore less time, therefore less Kraus operator influence.(hypothetical operator that applies error) Just like any other Mechanical machine we can use as many techniques like calibration to remove some error, but there are some errors that are unavoidable.

One common method to resolve issues with error is to run the function repetitively, of course this requires the machine to be somewhat functioning, and it takes up a lot of resources. The new Architecture we have designed does come in useful for this as well.

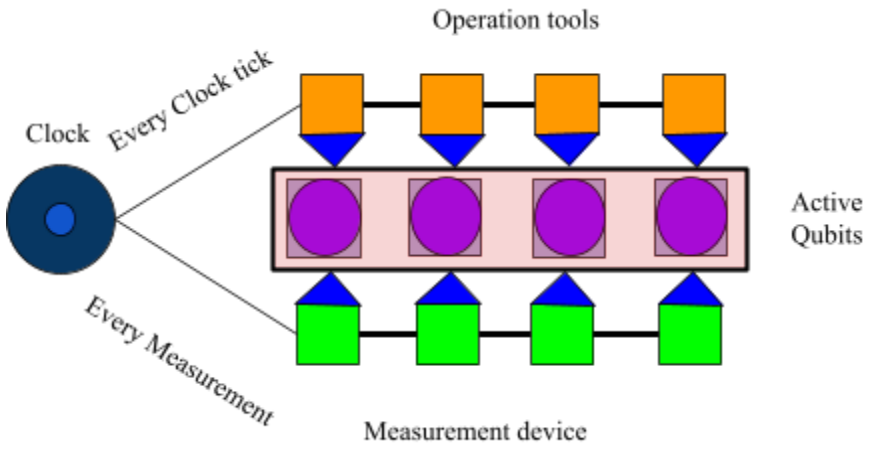


Diagram 2: This is the cycle which describes how the clock cycle will occur. Every clock cycle the Operation tools will be applied. When a qubit is measured the wave is destroyed, including the majority of information.

One common method to resolve issues with error is to run the function repetitively, of course this requires the machine to be somewhat functioning, and it takes up a lot of resources. The new Architecture we have designed does come in useful for this as well.

Let's take a look at Grover's Algorithm is an algorithm that takes a function(as an Oracle) and a goal value and attempts to find the input value for the function that creates the Goal value. Since our Inactive memory has already been made into an Oracle we can load the values of the function onto the Memory Sector as a list. If we run Algorithm just once we don't see anything special. If we run the Function multiple times at the same time we can see some advantages. Let's say the functional Oracle holds 128 qubits. If we were to run it 3 times regularly we would need 3 times the qubits, but since we are sharing the qubits we can use the same oracle for all of the function instances.

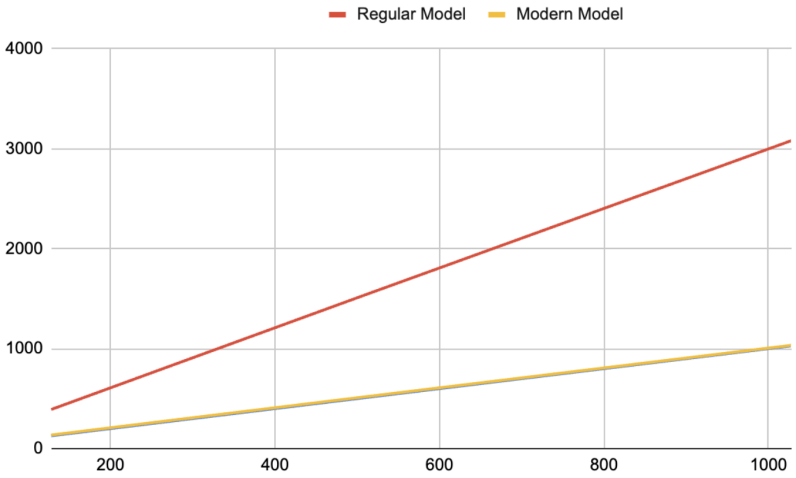


Diagram 3: This graph shows the Productivity of the Modern Model on Grover's Algorithm. The X-axis is the size of the Oracle and the Y axis is how many qubits is required to run the program assuming we are running three instances of the function simultaneously. These calculations were made by hand. This is important as Grover's Algorithm is important in many fields like Material Sciences etc.(IBM n.d.)

Additional Information and Proofs

[1]We want some proof that we can simulate the Pauli X gates simply by adjusting the external mechanism which notes which qubits the X gate has been applied to. We know that every quantum gate can be represented by a unitary matrix. Say we have some Unitary Matrix(We know that all quantum gates can be represented by a unitary matrix.) We say that there is some transformation, called U, which has been theorized. We can implement a X_n gate on the nth qubit. We now commit UX_n, we know that the unitary transformation of a unitary transformation is unitary therefore UX_n. Now we can do it in one transform. This removes the need for the X gate and SWAP gate the table shows its optimization.

NOTE: This doesn't mean that we need a whole new transformation operator for UX_n.. We could use other technology, this proof just states that the transformation would be possible

##Insert LATEX math, I was having a hard time importing it so I will add it to the final project##

Algorithm	Optimization per qubit(how many less transforms required to run algorithm) Just with the SWAP and X gates removed
Quantum Fourier Transform	Linear rate 1
Grover's Algorithm	Linear(exact rate depends on algorithm)
Shor's Algorithm for factoring	Linear rate 2(multiplied by error rate)

Refrences

De Wolf, Ronald Quantum computing Lecture Notes, 2019

Extra Image of entire setup

